

Guía de Ejercicios: Paralelismo y Computación Concurrente

Rendimiento, Ley de Amdahl, Ley de Gustafson, speedup y eficiencia

OpenMP y concurrencia — Sistemas Distribuidos y Paralelos

Departamento de Ingeniería Informática — Universidad de Santiago de Chile
Prof. Miguel Cárcamo

Semestre 1-2026

Introducción

Esta guía complementa la clase de **rendimiento en sistemas paralelos** (métricas secuenciales, FLOP/s, medición con reloj de pared vs. CPU, speedup, Amdahl, Gustafson, eficiencia y escalabilidad fuerte/débil). Los ejercicios conectan esas ideas con concurrencia (hilos) y OpenMP.

El objetivo es que puedan **declarar qué miden** ($T(1)$, $T(p)$, mismo algoritmo y misma entrada o no), **interpretar** S y E sin confundir strong/weak scaling, y relacionar órdenes $\mathcal{O}(\cdot)$ del trabajo con cuellos prácticos.

1. Ley de Amdahl, Ley de Gustafson y speedup

1.1. Conceptos fundamentales

Modelo CPU (repaso breve). Con I_c instrucciones dinámicas, CPI promedio y frecuencia f_{clk} :

$$T = \frac{I_c \cdot \text{CPI}}{f_{\text{clk}}}.$$

(**Nota:** en paralelismo usamos otra letra f para la fracción serial de Amdahl; no confundir con f_{clk} .)

Speedup habitual. Con p recursos paralelos:

$$S(p) = \frac{T(1)}{T(p)},$$

donde $T(1)$ y $T(p)$ son tiempos de *reloj de pared* de la misma referencia experimental, bien definidos.

Ley de Amdahl (strong scaling, problema fijo). Sea $f \in [0, 1]$ la fracción del tiempo en un procesador que no se paraleliza. Entonces

$$S(p) \leq \frac{1}{f + \frac{1-f}{p}} \xrightarrow{p \rightarrow \infty} \frac{1}{f}.$$

Ley de Gustafson (carga que escala). Con T_s tiempo de la parte no paralelizable y T_p de la parte paralelizable, ambos en un CPU,

$$S_G(p) = \frac{T_s + pT_p}{T_s + T_p}.$$

Una forma equivalente del *speedup escalado* es $S_G(p) = f + (1-f)p$ con la misma $f = T_s/(T_s + T_p)$.

Eficiencia. $E(p) = S(p)/p$.

Escalabilidad. *Strong scaling:* tamaño del problema fijo, crece p . *Weak scaling:* el problema crece con p de modo que la carga por procesador se mantiene; suele analizarse con la lectura de Gustafson.

Convención de notación

T_s y T_p en Amdahl/Gustafson son tiempos de **partes** del programa en un procesador. No son lo mismo que $T(1)$ y $T(p)$ del speedup global, salvo que usted fije explícitamente esa descomposición como referencia.

Ejemplo (fracción serial que cae con el tamaño). Sea tiempo secuencial total $T_{\text{tot}}(W) = 100 + W^2$ (parte fija 100 y parte $\mathcal{O}(W^2)$). La fracción de tiempo serial en un CPU es

$$f(W) = \frac{100}{100 + W^2}.$$

Para $W = 10$, $f(10) = 0,5$; para $W = 100$, $f(100) \approx 0,01$; si $W \rightarrow \infty$, $f(W) \rightarrow 0$: el cuello serial *relativo* se diluye (lectura típica en weak scaling / problemas grandes), mientras Amdahl con f fija acota el speedup con problema de tamaño fijo.

1.2. Ejercicios teóricos

Ejercicio 1. Amdahl y notación $T(1)$, $T(p)$

Un programa en la referencia de un CPU tiene fracción serial $f = 0,12$ (12 % del tiempo no paraleliza) y el resto es perfectamente paralelizable en ideal sin overhead.

- Escriba $S(p)$ según Amdahl y calcule $S(2)$, $S(4)$, $S(8)$, $S(16)$ y $S(32)$.
- ¿Cuál es $\lim_{p \rightarrow \infty} S(p)$?
- Si $T(1) = 150$ s y el modelo ideal de Amdahl aplica, ¿cuánto vale $T(8)$?
- Grafique $S(p)$ vs. p y comente la saturación respecto de $1/f$.

Ejercicio 2. Strong vs. weak y Amdahl vs. Gustafson

Un ordenamiento domina con $\mathcal{O}(N \log N)$ trabajo útil; inicialización y recogida de resultados suman un término $\mathcal{O}(1)$ en tiempo despreciable frente a N grande.

- Para $N = 2 \cdot 10^3$, suponga medido $f = 0,06$ (fracción serial del tiempo en un CPU). Calcule la cota de Amdahl $S(16)$.
- Para $N = 5 \cdot 10^6$, suponga $f = 0,002$. Calcule de nuevo $S(16)$ con Amdahl.
- Explique por qué comparar solo Amdahl con f fija *no* describe un experimento de weak scaling donde N crece con p . En una oración, relaciónelo con la ley de Gustafson.
- Con el f del ítem (a), calcule el *speedup escalado* de Gustafson $S_G(p) = f + (1-f)p$ para $p = 16$ (recuerde: es otra convención de reporte, no el mismo cociente $T(1)/T(p)$ con el mismo N que en (b)).

Ejercicio 3. Overhead lineal en p y eficiencia

Modelo didáctico (como en clase, con overhead agregado): tiempo de pared

$$T(p) = f T_{\text{ref}} + \frac{(1-f)T_{\text{ref}}}{p} + \alpha p,$$

con $T_{\text{ref}} = 120$ s, $f = 0,1$, $\alpha = 0,03$ s (costo creciente con p , p. ej. sincronización o contención).

- (a) Escriba $S(p) = T_{\text{ref}}/T(p)$ y $E(p) = S(p)/p$.
- (b) Calcule $S(p)$ y $E(p)$ para $p \in \{2, 4, 8, 16, 32\}$.
- (c) Estime por búsqueda o derivación el p que maximiza $S(p)$ en este modelo (puede no ser potencia de 2).
- (d) Comente cuándo $E(p) \ll 1$ y si tendría sentido reportar superlinealidad en este modelo idealizado.

Ejercicio 4. Matrices: orden del trabajo y escalabilidad

Sea $T(1, N) = 2N^3 + 80$ (unidades de tiempo; el término 80 modela fase serial fija). Paralelo ideal con overhead:

$$T(p, N) = \frac{2N^3}{p} + 80 + 6p.$$

- (a) Calcule $S(p) = T(1, N)/T(p, N)$ para $N = 120$ y $p \in \{4, 8, 16\}$.
- (b) Repita para $N = 900$ con los mismos p .
- (c) Discuta **strong scaling** fijando N y subiendo p : ¿en qué régimen domina $2N^3/p$ y cuándo domina $6p$?
- (d) Proponga un experimento de **weak scaling** (p. ej. N proporcional a p) y diga qué observaría en $T(p, N)$ si el término dominante del trabajo fuera $\Theta(N^3/p)$.

Ejercicio 5. Repaso: CPI y FLOP/s (opcional, corto)

Si $I_c = 8 \cdot 10^8$, CPI = 6 y $f_{\text{clk}} = 2,5 \text{ GHz}$, calcule T . Si ese programa ejecuta $4 \cdot 10^9$ FP en ese tiempo, ¿cuántos GFLOP/s reportaría?

Ejercicio 6. Lectura de time (opcional)

Un proceso imprime **real 40s, user 120s, sys 5s**. ¿Puede ser coherente en una máquina de 4 núcleos? Explique en una frase la diferencia entre **real** y **user**.

2. Concurrencia General

2.1. Conceptos Fundamentales

La concurrencia general se refiere a la capacidad de ejecutar múltiples tareas de forma simultánea, ya sea mediante procesos o hilos. Al medir speedups, use **reloj de pared** para $T(1)$ y $T(p)$ (p. ej. **time** del shell: **real**) y declare cuántos hilos o núcleos usó la máquina. Los conceptos clave incluyen:

- **Race conditions**: Condiciones de carrera cuando múltiples hilos acceden a recursos compartidos.
- **Sincronización**: Mecanismos como mutex, semáforos y barreras.
- **Deadlock**: Situación donde dos o más procesos se bloquean mutuamente.
- **Overhead de creación**: Costo de crear y gestionar hilos/procesos.

2.2. Ejercicios Prácticos

Ejercicio 7. Análisis de Speedup en Programas Concurrentes

Considere un programa que procesa M elementos independientes. La versión secuencial procesa cada elemento en t segundos. La versión concurrente crea n hilos, cada uno procesando aproximadamente $\frac{M}{n}$ elementos, pero introduce un overhead de sincronización de o segundos por hilo.

- (a) Derive la fórmula del speedup considerando el overhead.
- (b) Para $M = 1000$, $t = 0,1$ segundos y $o = 0,05$ segundos, calcule el speedup para $n = 2, 4, 8, 16$.
- (c) Identifique la fracción secuencial y paralelizable según la Ley de Amdahl.
- (d) Determine el número óptimo de hilos que maximiza el speedup.
- (e) Discuta cómo el overhead afecta la aplicabilidad de la Ley de Amdahl.

Ejercicio 8. Modelado de Overhead de Sincronización

Un programa concurrente tiene las siguientes características:

- Tiempo de cómputo puro: $T_{\text{comp}} = 100$ segundos (90 % paralelizable)
- Tiempo de sincronización: $T_{\text{sync}}(n) = 5 \log_2(n)$ segundos
- Tiempo de creación de hilos: $T_{\text{create}}(n) = 0,1 \cdot n$ segundos

- (a) Escriba la fórmula completa del tiempo de ejecución paralelo.
- (b) Calcule el speedup real para $n = 2, 4, 8, 16, 32$.
- (c) Compare con el speedup ideal de Amdahl (sin overhead).
- (d) Analice el punto donde el overhead supera los beneficios del paralelismo.

Ejercicio 9. Balanceo de Carga y Speedup

En un programa concurrente con n hilos, la carga puede estar desbalanceada. Suponga que:

- Hilo 1 procesa $w_1 = 60$ unidades de trabajo
- Hilos 2 a n procesan $w_i = 40$ unidades cada uno
- Cada unidad toma 1 segundo

- (a) Calcule el tiempo de ejecución paralelo considerando que todos los hilos deben terminar.
- (b) Calcule el speedup real vs. el speedup ideal (carga balanceada).
- (c) Derive una fórmula general para el speedup con desbalanceo.
- (d) Discuta cómo el desbalanceo afecta la Ley de Amdahl.

3. OpenMP

3.1. Conceptos Fundamentales

OpenMP es una API para programación paralela de memoria compartida que utiliza directivas del compilador para facilitar la paralelización de código. Los ejercicios relacionan regiones paralelas y scheduling con $T(1)/T(p)$, fracción serial efectiva y cuándo razonar con Amdahl (strong) frente a Gustafson (weak / problema creciente). Las directivas principales incluyen:

- `#pragma omp parallel`: Crea una región paralela
- `#pragma omp for`: Distribuye iteraciones de un bucle
- `#pragma omp sections`: Divide trabajo en secciones
- `#pragma omp critical`: Sección crítica para exclusión mutua
- Variables de entorno: `OMP_NUM_THREADS`, `OMP_SCHEDULE`

3.2. Ejercicios Prácticos

Ejercicio 10. Análisis de Código: Suma de Vectores

Analice el siguiente código OpenMP que suma dos vectores:

```
#include <omp.h>
void vector_sum(double *a, double *b, double *c, int n) {
    #pragma omp parallel for
    for (int i = 0; i < n; i++) {
        c[i] = a[i] + b[i];
    }
}
```

- Identifique la fracción paralelizable y secuencial según la Ley de Amdahl. ¿Cuál es el speedup máximo teórico con n hilos?
- Si el tiempo de ejecución secuencial es $T_s = 1,0$ segundo para $N = 10^6$ elementos, y el overhead de creación de región paralela es $O = 0,001$ segundos, calcule el speedup real para 2, 4, 8 y 16 hilos.
- ¿Por qué el speedup real puede ser menor que el teórico de Amdahl? Identifique al menos tres factores.
- Si el tamaño del vector aumenta a $N = 10^8$, ¿cómo cambia la fracción secuencial? ¿Qué ley (Amdahl o Gustafson) es más apropiada para analizar este caso?

Ejercicio 11. Análisis de Código: Problemas de Paralelización

Analice el siguiente código y responda:

```
double sum = 0.0;
#pragma omp parallel for
for (int i = 0; i < n; i++) {
    sum += array[i];
}
```

- ¿Qué problema tiene este código? Explique el concepto de *race condition*.
- ¿Cuál sería el speedup esperado si este código se ejecutara sin corregir el problema? Justifique.
- Proponga dos soluciones diferentes usando OpenMP y explique cómo cada una afecta el overhead y, por tanto, el speedup.
- Si el overhead de sincronización de una solución es $O_1 = 0,01 \cdot n$ y de la otra es $O_2 = 0,05 \cdot \log_2(n)$, calcule el speedup para cada solución con $n = 8$ hilos y $N = 10^6$ elementos (tiempo secuencial = 1.0s).

Ejercicio 12. Análisis de Overhead en OpenMP

Un bucle paralelo con OpenMP tiene:

- $N = 1,000,000$ iteraciones
- Tiempo por iteración: $t_i = 1$ microsegundo
- Overhead de creación de región paralela: $O_{\text{fork}} = 100$ microsegundos
- Overhead por iteración (scheduling): $O_{\text{iter}} = 0,01$ microsegundos

- Calcule el tiempo total para la versión secuencial.

- (b) Calcule el tiempo total para la versión paralela con n hilos.
- (c) Derive la fórmula del speedup considerando overheads.
- (d) Calcule el speedup para $n = 2, 4, 8, 16$ y compare con Amdahl.
- (e) Determine el tamaño mínimo de N para que el paralelismo sea beneficioso con $n = 4$.

Ejercicio 13. Análisis de Código: Producto Matriz-Vector

Analice el siguiente código OpenMP para el producto matriz-vector $y = Ax$:

```
void matrix_vector_mult(double **A, double *x, double *y, int m, int n) {
    #pragma omp parallel for schedule(static)
    for (int i = 0; i < m; i++) {
        y[i] = 0.0;
        for (int j = 0; j < n; j++) {
            y[i] += A[i][j] * x[j];
        }
    }
}
```

- (a) Identifique qué bucle está paralelizado y explique por qué esta es una buena estrategia.
- (b) Si el tiempo secuencial es $T_s(m, n) = 2mn$ microsegundos, calcule el speedup teórico según Amdahl para una matriz 1000×1000 con 8 hilos (asuma fracción secuencial despreciable).
- (c) Considere que el overhead de creación de región paralela es $O = 100$ microsegundos. Calcule el speedup real para matrices de tamaño 100×100 , 1000×1000 y 10000×10000 con 8 hilos.
- (d) Analice cómo el tamaño de la matriz afecta el speedup. ¿Qué ley (Amdahl o Gustafson) describe mejor este comportamiento? Justifique.
- (e) Si cambiáramos a `schedule(dynamic)`, ¿cómo afectaría esto al overhead y al speedup? Explique.

Ejercicio 14. Análisis Comparativo: Estrategias de Paralelización

Se tienen dos versiones paralelas del mismo algoritmo. La versión A paraleliza el bucle externo y la versión B usa `collapse(2)` para paralelizar ambos bucles anidados:

```
// Versión A
#pragma omp parallel for
for (int i = 0; i < m; i++) {
    for (int j = 0; j < n; j++) {
        // trabajo O(1)
    }
}

// Versión B
#pragma omp parallel for collapse(2)
for (int i = 0; i < m; i++) {
    for (int j = 0; j < n; j++) {
        // trabajo O(1)
    }
}
```

- (a) ¿Cuántas iteraciones paralelas tiene cada versión? (Versión A: m , Versión B: mn)

- (b) Si $m = 100$ y $n = 1000$, y tenemos 8 hilos, analice el balanceo de carga en cada versión.
- (c) El overhead de scheduling en la versión A es $O_A = 0,1 \cdot m$ y en B es $O_B = 0,1 \cdot mn$. Para $m = 100$, $n = 1000$ y tiempo de trabajo total $T = 1,0s$, calcule el speedup de cada versión.
- (d) ¿Bajo qué condiciones (valores de m y n) una versión es mejor que la otra? Justifique considerando overhead y balanceo de carga.

Ejercicio 15. Análisis de Código: Reducción vs. Sección Crítica

Analice las dos versiones paralelas para calcular π mediante integración numérica:

$$\pi = 4 \int_0^1 \frac{1}{1+x^2} dx \approx \frac{4}{N} \sum_{i=0}^{N-1} \frac{1}{1 + (\frac{i+0,5}{N})^2}$$

Versión A: Usando reduction

```
double sum = 0.0;
#pragma omp parallel for reduction(+:sum)
for (int i = 0; i < N; i++) {
    double x = (i + 0.5) / N;
    sum += 1.0 / (1.0 + x * x);
}
double pi = 4.0 * sum / N;
```

Versión B: Usando sección crítica

```
double sum = 0.0;
#pragma omp parallel
{
    double local_sum = 0.0;
    #pragma omp for
    for (int i = 0; i < N; i++) {
        double x = (i + 0.5) / N;
        local_sum += 1.0 / (1.0 + x * x);
    }
    #pragma omp critical
    {
        sum += local_sum;
    }
}
double pi = 4.0 * sum / N;
```

- (a) Identifique las diferencias entre ambas versiones en términos de sincronización.
- (b) Si el tiempo de cómputo por iteración es $t = 1$ microsegundo y el overhead de **reduction** es $O_{\text{red}} = 0,01 \cdot n$ microsegundos (donde n es el número de hilos), mientras que el overhead de **critical** es $O_{\text{crit}} = 0,1 \cdot n$ microsegundos, calcule el tiempo total para cada versión con $N = 10^6$ y $n = 8$ hilos.
- (c) Calcule el speedup de cada versión vs. la versión secuencial (tiempo secuencial = $N \cdot t$).
- (d) Modele la fracción secuencial según Amdahl para cada versión. ¿Cuál tiene menor fracción secuencial?
- (e) Para $N = 10^4, 10^5, 10^6, 10^7$, analice cómo cambia el speedup de cada versión. ¿Qué ley (Amdahl o Gustafson) describe mejor este comportamiento?

Ejercicio 16. Sections y Tasking

Un programa tiene tres tareas independientes con tiempos $T_1 = 50\text{s}$, $T_2 = 30\text{s}$, $T_3 = 20\text{s}$.

- (a) Calcule el tiempo secuencial total.
- (b) Implemente usando `#pragma omp sections` con 3 hilos.
- (c) Calcule el speedup teórico y real.
- (d) Generalice para k tareas con tiempos $T_1, T_2, \dots, T_k$ y n hilos.
- (e) Discuta cómo este caso se relaciona con la Ley de Amdahl cuando las tareas tienen diferentes duraciones.

4. Ejercicios Integradores

Los siguientes ejercicios integran múltiples conceptos y tecnologías:

Ejercicio 17. Comparación de Tecnologías: Suma de Vectores

Implemente la suma de vectores $c = a + b$ usando:

- Versión secuencial en C
 - Concurrencia con pthreads
 - OpenMP
- (a) Mida el speedup para cada tecnología con $N = 10^4, 10^5, 10^6, 10^7$ elementos.
 - (b) Identifique la fracción paralelizable y secuencial para cada implementación.
 - (c) Calcule el speedup teórico según Amdahl y compare con los resultados reales.
 - (d) Analice los diferentes tipos de overhead en cada tecnología:
 - Creación de hilos/procesos
 - Sincronización
 - Falsos compartidos y jerarquía de caché (si aplica)
 - Scheduling (OpenMP) y reparto de carga
 - (e) Determine para qué tamaños de problema cada tecnología es más apropiada.
 - (f) Discuta cómo las leyes de Amdahl y Gustafson se manifiestan de manera diferente en cada tecnología.

Ejercicio 18. Análisis de Speedup en Algoritmo Complejo

Considere un algoritmo de procesamiento de imágenes que:

1. Lee una imagen ($T_{\text{read}} = 10$ ms, secuencial)
 2. Aplica un filtro a cada píxel ($T_{\text{filter}} = 100$ ms, 100 % paralelizable)
 3. Realiza una reducción para calcular estadísticas ($T_{\text{reduce}} = 20$ ms, 90 % paralelizable)
 4. Escribe el resultado ($T_{\text{write}} = 10$ ms, secuencial)
- (a) Calcule el tiempo secuencial total.
 - (b) Implemente versiones paralelas usando OpenMP (p. ej. paralelizar el filtro y la reducción con distintas estrategias).
 - (c) Para OpenMP, calcule el speedup teórico según Amdahl y el speedup real medido con $T(1)/T(p)$.

- (d) Modele un costo fijo de E/S al leer/escribir la imagen ($T_{\text{read}}, T_{\text{write}}$) y discuta cómo ese término fijo afecta la fracción serial vista por Amdahl al aumentar la resolución.
- (e) Analice cómo la Ley de Gustafson se aplica cuando se procesan imágenes de diferentes resoluciones (weak scaling: más píxeles cuando hay más recursos).
- (f) Compare los speedups obtenidos y discuta limitaciones prácticas (granularidad, sincronización).

Ejercicio 19. Modelado Teórico de Speedup

Modele teóricamente el speedup de un algoritmo genérico que tiene:

- Fracción secuencial inicial: $s_1 = 0,1$
 - Fracción paralelizable: $p = 0,8$
 - Fracción secuencial final: $s_2 = 0,1$
 - Overhead de paralelización: $O(n) = \alpha \cdot n + \beta \cdot \log(n)$
- (a) Derive la fórmula completa del speedup.
 - (b) Para $\alpha = 0,01$, $\beta = 0,1$, calcule el speedup para $n = 2, 4, 8, 16, 32, 64$.
 - (c) Compare con el speedup ideal de Amdahl (sin overhead).
 - (d) Analice el impacto de cada componente del overhead.
 - (e) Determine el número óptimo de procesadores.
 - (f) Discuta cómo distintos entornos (p. ej. OpenMP frente a pthreads) pueden implicar distintos α y β en el modelo de overhead.

Ejercicio 20. Benchmarking y Validación de Leyes

Diseñe un conjunto de benchmarks que permitan validar empíricamente las leyes de Amdahl y Gustafson.

- (a) Seleccione 3 algoritmos con diferentes características:
 - Algoritmo 1: Alta fracción paralelizable (95 %), tamaño fijo
 - Algoritmo 2: Fracción paralelizable media (70 %), tamaño variable
 - Algoritmo 3: Baja fracción paralelizable (50 %), tamaño grande
- (b) Implemente versiones secuenciales y paralelas (OpenMP y/o pthreads).
- (c) Mida el speedup para diferentes números de procesadores/hilos.
- (d) Ajuste los modelos de Amdahl y Gustafson a los datos experimentales.
- (e) Calcule los parámetros (fracción secuencial, overhead) que mejor explican los resultados.
- (f) Analice las desviaciones entre teoría y práctica.
- (g) Proponga mejoras a los modelos teóricos basadas en los resultados.

Notas Finales

Esta guía de ejercicios está diseñada para que los estudiantes alcancen los siguientes objetivos de aprendizaje:

- Comprender las limitaciones teóricas del paralelismo establecidas por la Ley de Amdahl.
- Reconocer cuándo la Ley de Gustafson es más apropiada para analizar problemas escalables.

- Aplicar estas leyes para predecir y analizar el rendimiento de programas paralelos.
- Identificar y cuantificar los diferentes tipos de overhead presentes en cada tecnología.
- Seleccionar la tecnología más apropiada según las características del problema a resolver.
- Validar empíricamente las leyes teóricas mediante experimentación práctica.

Se recomienda encarecidamente que los estudiantes implementen y midan los ejercicios prácticos para obtener experiencia real con las diferentes tecnologías y validar su comprensión de los conceptos teóricos. La combinación de análisis teórico y experimentación práctica es fundamental para desarrollar una comprensión profunda del paralelismo y la computación concurrente.